

Rank-based Tests in MLR - Examples in R

B.A. Hartlaub and A.S. Wagaman

MLR Rank-Based Test Examples

Example 1

Researchers observed the following data on 20 individuals with high blood pressure:

- blood pressure (*BP*, in mm Hg)
- age (*Age*, in years)
- weight (*Weight*, in kg)
- body surface area (*BSA*, in m^2)
- duration of hypertension (*Dur*, in years)
- basal pulse (*Pulse*, in beats per minute)
- stress index (*Stress*)

Our goal is to build a model for blood pressure as a function of (some subset) of the other variables.

```
BP <- read.csv("https://awagaman.people.amherst.edu/nonparametricstats/bloodpress.csv")
```

The basic `rfit` summary includes an overall drop test option that is similar to the nested F-test for parametric models. When performing a drop test, the null hypothesis is that the reduced model is sufficient. The alternative hypothesis is that the full model (at least one variable is useful) is better. In other words, the null is that the slopes for predictors in the full model but not reduced are all 0, and the alternative is that the slope of at least one of those predictors is not zero. There is another option for the overall test, called the Wald test. Here is the code for these two options with the full model.

```
# the . in the command tells it to use all the non-response variables as predictors
fullmodel <- rfit(BP ~ ., data = BP)
summary(fullmodel, overall.test = 'drop') # default seems to be drop-test approach
```

```
## Call:
## rfit.default(formula = BP ~ ., data = BP)
##
## Coefficients:
##           Estimate Std. Error t.value    p.value
## (Intercept) -12.0441111    2.6529548  -4.5399 0.0005553 ***
## Age           0.6866014    0.0514734  13.3390 5.819e-09 ***
## Weight        0.9725548    0.0654844  14.8517 1.560e-09 ***
## BSA           3.6166943    1.6396404   2.2058 0.0460060 *
## Dur           0.0762141    0.0502652   1.5162 0.1533927
## Pulse        -0.0847368    0.0535520  -1.5823 0.1375897
## Stress        0.0067336    0.0035408   1.9017 0.0795976 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.9729667
## Reduction in Dispersion Test: 77.98147 p-value: 0
```

```
# in case you are interested, here is the alternative offered
summary(fullmodel, overall.test = 'wald')
```

```
## Call:
## rfit.default(formula = BP ~ ., data = BP)
##
## Coefficients:
##           Estimate Std. Error t.value    p.value
## (Intercept) -12.0441111    2.6529548  -4.5399 0.0005553 ***
## Age           0.6866014    0.0514734  13.3390 5.819e-09 ***
## Weight        0.9725548    0.0654844  14.8517 1.560e-09 ***
## BSA           3.6166943    1.6396404   2.2058 0.0460060 *
## Dur           0.0762141    0.0502652   1.5162 0.1533927
## Pulse        -0.0847368    0.0535520  -1.5823 0.1375897
## Stress        0.0067336    0.0035408   1.9017 0.0795976 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## Overall Wald Test: 516.0713 p-value: 0
```

Using the drop-test output, we see the test statistic is 77.98 and p-value is approximately 0.

What should we conclude?

Note that the output using the Wald Test statistic and p-value would have provided the same conclusion. A nice option that comes with the `overall.test = 'drop'` (default) option is that we get a robust version of R-squared that you can use to discuss model fit.

What if we wanted to use the `drop.test` command to compare models? Well, here are some other models we could consider, using different predictor subsets.

```
fm3 <- rfit(BP ~ Weight + Age + Dur + Stress, data = BP)
fm4 <- rfit(BP ~ Weight + Age, data = BP)
```

Remember that the null hypothesis is that the reduced model is sufficient, and this indicates the coefficients for predictors in the full model but not the reduced model all have slopes of 0. Note that the full model goes in the command first (which is the opposite of using the `anova` command for the nested F test).

If we test this below, what are we testing? What is the result?

```
drop.test(fullmodel, fm3)
```

```
##
## Drop in Dispersion Test
## F-Statistic    p-value
##    7.4543896    0.0069713
```

We are testing the null hypothesis that the coefficients associated with BSA and Pulse are equal to 0 against the alternative hypothesis that at least one of these coefficients is non-zero. Since the p-value is well below 0.05, we reject the null hypotheses and conclude that the full model is better than the first reduced model (fm3).

What about here?

```
drop.test(fm3, fm4)
```

```
##
## Drop in Dispersion Test
## F-Statistic    p-value
##    1.12703     0.34995
```

When comparing the two reduced models (fm3 and fm4), we are testing the null hypothesis that the two coefficients associated with Duration and Stress are equal to zero. Since the p-value 0.34995 is well above 0.05, we cannot reject the null hypothesis. Thus, we conclude that fm4 would be a better model than fm3, based on this drop in deviance test.

You can use the *summary* command for these models in order to get coefficients, write down regression equations, etc. Remember to put `overall.test = 'drop'` in the summary command (it should work as its the default, but if not, be sure you get the drop-test output).

The summary output also gives you a test statistic and p-value for each individual predictor. Remember to interpret these as relating to the significance of that predictor while accounting for the presence of the other variables in the model.

Example with a Categorical predictor

To consider how rfit works with a categorical predictor, we briefly turn to the iris data set in R.

```
data(iris)
irisfull <- rfit(Sepal.Length ~ ., data = iris)
summary(irisfull, overall.test = 'drop')
```

```
## Call:
## rfit.default(formula = Sepal.Length ~ ., data = iris)
##
## Coefficients:
##              Estimate Std. Error t.value    p.value
## (Intercept)    2.185396   0.291616   7.4941 6.214e-12 ***
## Sepal.Width     0.481741   0.089569   5.3785 2.967e-07 ***
## Petal.Length    0.856741   0.071313  12.0138 < 2.2e-16 ***
## Petal.Width    -0.337077   0.157342  -2.1423  0.033849 *
## Speciesversicolor -0.764047  0.249932  -3.0570  0.002665 **
## Speciesvirginica -1.095227  0.347292  -3.1536  0.001963 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.7725931
## Reduction in Dispersion Test: 97.84526 p-value: 0
```

```
irissubset <- rfit(Sepal.Length ~ Sepal.Width*Species, data = iris)
summary(irissubset, overall.test = 'drop')
```

```
## Call:
## rfit.default(formula = Sepal.Length ~ Sepal.Width * Species,
##              data = iris)
##
## Coefficients:
##              Estimate Std. Error t.value    p.value
## (Intercept)    2.73333    0.54328   5.0312 1.431e-06 ***
## Sepal.Width     0.66667    0.15752   4.2321 4.102e-05 ***
## Speciesversicolor 0.65588    0.75922   0.8639  0.3891
## Speciesvirginica 1.00852    0.77576   1.3000  0.1957
## Sepal.Width:Speciesversicolor 0.24296    0.24703   0.9835  0.3270
## Sepal.Width:Speciesvirginica 0.27682    0.24310   1.1387  0.2567
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.6690924
```

```
## Reduction in Dispersion Test: 58.23335 p-value: 0
```

rfit seems to behave exactly as expected for categorical variables. This includes the ability to include interactions. Indicator variables are created correctly for the categorical variable levels. Be sure you can tell what the reference level is!

For the overall drop test, we see a test statistic of 97.845 and a p-value reported as 0. We can reject the null hypothesis and state that it appears at least one predictor has a significant non-zero slope that is useful for predicting Sepal.Length.

Should you try to compare the irisfull and irissubset models here? If not, what would an appropriate model to compare be?

No, the predictor variables in the reduced model are not a subset of those in the full model. One possibility is to compare the iris subset model (irissubset) with another model that does not have the interaction terms. Another possibility is to compare the full model (irisfull) with the reduced model that does not include interaction terms. Give this a try!

Example with no relationship

We generate toy data here for this example.

```
set.seed(225)
x <- rnorm(100, 25, 2)
y <- rnorm(100, 15, 6)
test <- rfit(y ~ x)
summary(test, overall.test = 'drop')
```

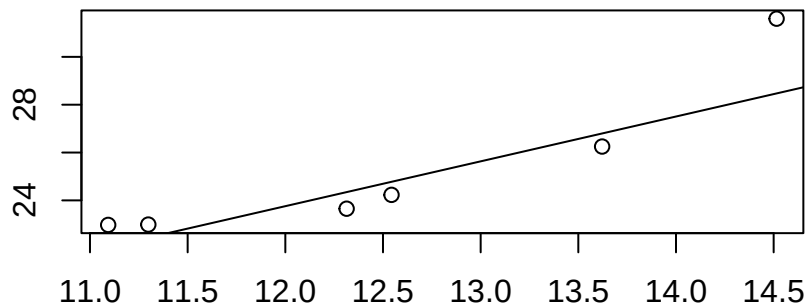
```
## Call:
## rfit.default(formula = y ~ x)
##
## Coefficients:
##             Estimate Std. Error t.value p.value
## (Intercept) 18.57840    7.19104  2.5835 0.01126 *
## x           -0.19030    0.28124 -0.6767 0.50022
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.004721756
## Reduction in Dispersion Test: 0.46493 p-value: 0.49694
```

This is just an example to see what happens if you don't have significance. X and Y are generated independently, so if the p-value was significant, that's a Type I error. You can change the seed to examine multiple data sets.

Comparing Rfit and Theil

We should be able to fit SLR with rfit, so let's check and see what happens.

```
# prior 6 data points example
x <- c(12.54367, 11.09313, 13.62189, 12.31389, 14.51518, 11.29860)
y <- c(24.23092, 22.97545, 26.25129, 23.65294, 31.59527, 22.98974)
mydata <- data.frame(x, y)
with(mydata, theil(x, y, beta.0 = 0, type = "t"))
```



```
## Alternative: beta not equal to 0
## C = 15, C.bar = 1, P = 0.003
## beta.hat = 1.874
## alpha.hat = 1.272
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 0.555, 3.735
```

```
rfitmod <- rfit(y ~ x, data = mydata)
summary(rfitmod, overall.test = 'drop')
```

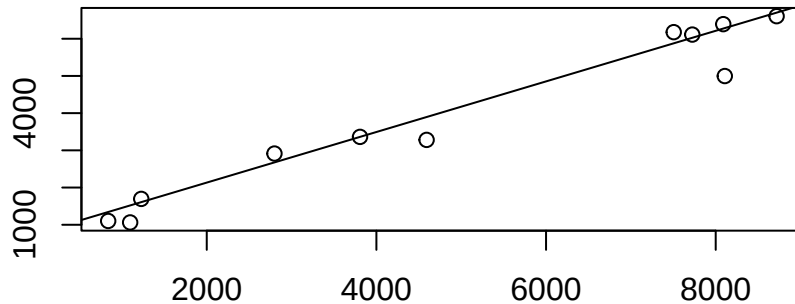
```
## Call:
## rfit.default(formula = y ~ x, data = mydata)
##
## Coefficients:
##           Estimate Std. Error t.value p.value
## (Intercept) -0.071078  10.669528  -0.0067 0.99500
## x           1.986506   0.842980   2.3565 0.07796 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.6487894
## Reduction in Dispersion Test: 7.38918 p-value: 0.05308
```

rfit is providing different results than theil, as the approach is different, so we recommend that you should use theil for nonparametric SLR, but rfit for nonparametric MLR. Again, rfit uses a different optimization criterion via a rank-based approach compared to theil (median-based approach), so this makes sense. Here's another quick example to illustrate the difference.

Do flexible work schedules reduce the demand for resources?

The Lake County, Illinois, Health Department experimented with a flexible four-day workweek. For a year, the department recorded the mileage (and other variables) driven by 11 field workers on an ordinary five-day workweek. Then it changed to a flexible four-day workweek and recorded mileage (and other variables) for a year for the same workers in that setting.

```
# mileage example
day5 <- c(2798, 7724, 7505, 838, 4592, 8107, 1228, 8718, 1097, 8089, 3807) # use as X
day4 <- c(2914, 6112, 6177, 1102, 3281, 4997, 1695, 6606, 1063, 6392, 3362)
daydata <- data.frame(day5, day4)
with(daydata, theil(day5, day4, beta.0 = 0, type = "t"))
```



```
## Alternative: beta not equal to 0
## C = 43, C.bar = 0.782, P = 0
## beta.hat = 0.68
## alpha.hat = 773.404
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 0.471, 0.762
```

```
mod <- rfit(day4 ~ day5, data = daydata)
summary(mod, overall.test = 'drop')
```

```
## Call:
## rfit.default(formula = day4 ~ day5, data = daydata)
##
## Coefficients:
##           Estimate Std. Error t.value    p.value
## (Intercept) 722.900574 330.735264  2.1857  0.05664 .
## day5         0.693223   0.040675 17.0430 3.705e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.9108481
## Reduction in Dispersion Test: 91.95126 p-value: 1e-05
```

The difference here isn't as extreme, but there is a difference. This means not only can you examine the difference between parametric and nonparametric approaches, but there are multiple nonparametric approaches you can consider. You can examine the difference in models between parametric SLR, nonparametric SLR with a median-based approach and nonparametric SLR with a rank-based approach!

Data Sources

Blood pressure data set from Penn State's STAT 501 course. Referenced online at <https://online.stat.psu.edu/stat501/lesson/12/12.1> Data of last access: July 25, 2025.

Iris data. Available in R with `data(iris)`. Original sources referenced in R help:

Fisher, R. A. (1936) The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7, Part II, 179–188. doi:10.1111/j.1469-1809.1936.tb02137.x.

Anderson, Edgar (1935). The irises of the Gaspé Peninsula, *Bulletin of the American Iris Society*, 59, 2–5.

Simulated data for the Toy example (6 data points).

Charles S. Catlin, “Four-day Work Week Improves Environment,” *Journal of Environmental Health*, Denver, 59:7, 1997.

Standard Reference

These materials were created for use in an undergraduate nonparametrics course. Where provided, textbook references refer to *Nonparametric Statistical Methods* (3rd Edition) by Hollander, Wolfe, and Chicken. Notation is chosen to be consistent with that text. However, materials may be used independently of the textbook.

License

This work is protected by creative commons license CC BY-NC-SA.